

TECHNICAL REPORT

FPU pipeline depth: performance against area

openhwgroup/cv32e40p

Document	RVX-CV32E40P-TRL5-01
Record date	2026-10-03
Prepared by	Rivoryxa Technologies
Classification	Public technical note. Not a verification sign-off.
Website	www.rivoryxatechnologies.com

1 Answer

- Without register retiming, the extra FPU pipeline stages only cost. F1 and F2 take more cycles and the clock does not improve, because the stage registers sit at the FPU output (PipeConfig = AFTER) and the critical path stays inside the FPU.
- With retiming, the extra stages pay off clearly. One stage (F1) roughly halves the core's worst register-to-register path (32.3 -> 17.1 ns). Two stages (F2) bring it to 13.1 ns, close to the core without an FPU.
- Cycle cost: F1 costs 0 to 6% more cycles on five small FP kernels and 30% on the upstream matmul_32b_float program. F2 costs 1.5 to 28.5%, and 64% on that program.
- Performance per area (geometric mean over six FP workloads, F0 without retiming = 1.00):
 - Without retiming: F1 0.90, F2 0.78.
 - With retiming: F0 1.02, F1 1.81, F2 2.00.
 - F2's lead over F1 is small on code with back-to-back FP operations: 1.6% on axpy and 4% on the upstream matmul.
- The Zfinx configurations (Z0, Z1, Z2) are about 18% smaller than F at every latency and follow the same pattern.

These are synthesis estimates on an open 130 nm library without place and route. Read the ratios, not the absolute numbers.

2 The question

At the CVE4 meeting of 18 September 2026, Mike Thompson (OpenHW) asked which FPU pipeline configuration should go to TRL5, and what each option costs. The group's position on performance against area is still open. This report puts cycles, clock period and area for each configuration side by side.

3 What we did

1. Cycles. We wrote six bare-metal C kernels. Each reads mcycle and minstret around the kernel and checks its result against a checksum computed on the host. We ran them on the Verilator core testbench for every configuration. We also ran the testbench's own matmul_32b_float program (built -O3 by its own flags) on the same models.
2. Area and delay. We synthesised the whole core (cv32e40p_top) with Yosys and yosys-slang, mapped it to sky130_fd_sc_hd at the typical corner, and timed it with OpenSTA. We did this once plain, and once with abc register retiming (abc -dff).
3. Combined. Execution time = cycles x worst register-to-register path. Performance per area = 1 / (time x area), normalised to F0 plain.

We wrote our predictions down before the first run ([PREDICTIONS.md](#)). Every F1 change, the integer control kernel, the instruction counts and the retimed ordering came out as predicted. Three predictions missed; they are explained under Results.

config	FPU	FPU_ADDMUL_LAT = FPU_OTHERS_LAT	ZFINX
I0	0	-	0
F0 / F1 / F2	1	0 / 1 / 2	0
Z0 / Z1 / Z2	1	0 / 1 / 2	1

4 Results

Cycles

All 37 programs pass, and every checksum matches the host reference. instret is identical across latencies, so the extra cycles are stalls only. The integer-only control kernel takes 89,166 cycles on all seven configurations (I0 included). Deltas are against latency 0 of the same family.

kernel	what it stresses	F0	F1	F2	Z0	Z1	Z2
matmul 16x16	long dependent chain per output	34,960	+0	+512 (+1.5%)	34,961	+0	+256 (+0.7%)
dot (1024)	dependent fmadd accumulation	4,371	+256 (+5.9%)	+770 (+17.6%)	4,371	+256 (+5.9%)	+769 (+17.6%)
axpy (1024)	independent fmadd	5,394	+0	+1,536 (+28.5%)	5,394	+0	+1,536 (+28.5%)
FIR 16 taps	dependent chain per output	35,086	+0	+512 (+1.5%)	35,087	+0	+256 (+0.7%)
norm (128 vectors)	fdiv, fsqrt	6,925	+256 (+3.7%)	+1,152 (+16.6%)	6,925	+256 (+3.7%)	+1,152 (+16.6%)
upstream matmul_32b_float (-O3)	compiler-scheduled matmul	10,075	+3,072 (+30.5%)	+6,401 (+63.5%)	10,652	+768 (+7.2%)	+4,352 (+40.9%)
control (integer only)	none	89,166	+0	+0	89,166	+0	+0

Why the cycle cost looks like this. The RTL that sets it is cv32e40p_decoder.sv (apu_lat = LAT + 1) and cv32e40p_apu_disp.sv.

- F1: costs one cycle only when the very next instruction uses the FP result. Independent FP operations still overlap.
- F2: puts every FP operation in the same "multicycle" class as fdiv, so FP operations never overlap: each back-to-back FP operation costs about 2 extra cycles.
- Division and square root cost the same in every configuration.
- The compiler's schedule matters as much as the hardware. The -O3 F build of the upstream program leaves 12 next-instruction consumers per output: 12 x 256 outputs = 3,072 cycles, exactly the measured cost. The Zfinx build is short of registers and leaves only 3 (768 cycles).

Where our predictions missed:

1. matmul and FIR on F2 cost +512, not the +256 we predicted. The fmv.w.x that zeroes the accumulator collides with the next integer mv in write-back. The Zfinx build uses li instead and shows exactly +256. We inferred this from that difference; we did not trace it in a waveform.
2. The upstream program's F cost (+3,072 / +6,401) is far above its Z cost (+768 / +4,352), for the register-pressure reason above.
3. The Zfinx area saving (about 59,000 μm^2) came out larger than we predicted (15,000 to 50,000 μm^2).

Area (whole core, sky130_fd_sc_hd, typical corner, μm^2)

Compare within one column. The retimed flow models asynchronous resets as synchronous, so its areas are not comparable with the plain column.

config	plain	vs F0	retimed	vs F0 retimed
I0	169,912		164,465	
F0	320,487		299,909	
F1	325,566	+1.6%	313,762	+4.6%
F2	330,160	+3.0%	328,491	+9.5%
Z0	261,492	-18.4%	247,155	-17.6%

config	plain	vs F0	retimed	vs F0 retimed
Z1	267,069	-16.7%	260,345	-13.2%
Z2	271,540	-15.3%	272,580	-9.1%

Correction to the figures we shared on 21 September: the "+1.6% / +3.0%" area figures are from the plain flow, which gives no clock gain. The configuration that actually gets the clock gain (the retimed one) costs +4.6% / +9.5%.

Worst register-to-register path (OpenSTA, ns)

	I0	F0	F1	F2	Z0	Z1	Z2
plain	13.89	32.29	33.38	33.32	34.87	29.94	32.70
retimed	11.76	33.89	17.12	13.12	36.79	17.86	12.60

The plain Z0 and Z1 paths differ from F by +8% and -10%. Differences of this size between near-identical netlists also appear between F1 and F2 plain; treat the plain column as "no change".

Execution time (us = cycles x path)

kernel	F0 plain	F1 plain	F2 plain	F0 retimed	F1 retimed	F2 retimed	Z2 retimed
matmul	1,128.8	1,166.9	1,182.0	1,184.8	598.6	465.2	443.6
dot	141.1	154.4	171.3	148.1	79.2	67.4	64.7
axpy	174.2	180.0	230.9	182.8	92.4	90.9	87.3
FIR	1,132.9	1,171.1	1,186.2	1,189.0	600.7	466.9	445.2
norm	223.6	239.7	269.1	234.7	122.9	105.9	101.7
upstream matmul	325.3	438.8	549.0	341.4	225.1	216.1	189.0
control (integer)	2,879.1	2,976.2	2,971.2	3,021.8	1,526.6	1,169.5	1,123.2

Without retiming, F0 is fastest on every kernel. With retiming, F2 is fastest on every kernel.

Performance per area (1 / (time x area), F0 plain = 1.00)

	F0	F1	F2	Z0	Z1	Z2
plain, geometric mean of 6 FP workloads	1.00	0.90	0.78	1.12	1.25	0.99
retimed, geometric mean of 6 FP workloads	1.02	1.81	2.00	1.13	2.15	2.56
retimed, axpy (worst case for F2)	1.02	1.93	1.87	1.14	2.23	2.35
retimed, upstream matmul	1.02	1.48	1.47	1.08	1.96	2.02

5 What this means for TRL5

- The decision turns on the implementation flow, not only on the RTL. If the flow retimes registers into the FPU datapath, F1 gives about 1.8x the performance per area of F0, and F2 adds a little more on average. If it does not, F0 is the best choice and the extra stages are pure cost.
- F1 is the safer middle choice. It loses almost no cycles on dependent code and keeps most of the clock gain. F2 gains more clock but serialises FP operations, so its margin over F1 depends on the code (from -3% to +23% per area across these workloads).
- The cycle cost of F1 and F2 depends on the compiler. A schedule that keeps the next instruction independent of an FP result would remove most of it. That is a software lever that costs no area.
- Zfinx is about 18% smaller at every latency. If the TRL5 configurations include Zfinx, Z1 and Z2 give the best performance per area here.
- #1060 does not change this picture. Both fixes hold on F1 and F2 (see [report 02](#)).

6 How to repeat

Tools: Verilator 5.050, xPack riscv-none-elf-gcc 15.2, Yosys 0.69 with the yosys-slang plugin (OSS CAD Suite), OpenSTA 3.1.0, and the sky130_fd_sc_hd liberty file sky130_fd_sc_hd__tt_025C_1v80.lib (source commit and sha256 in [synth.sh](#)). The testbench is cv32e40p-dv-review 89ad543 plus [tb_fpu_config_plumbing.patch](#); without

that patch every build runs at latency 0.

From the root of this repository:

```
# cycles cd tests/cv32e40p-fpu-perf && ./build.sh && cd ../cv32e40p-tb-run for c in F0 F1 F2; do ./build_tb.sh
$c; ./run_elfs.sh work/$c/obj/Vtb_top ../cv32e40p-fpu-perf/build/f.elf; done for c in Z0 Z1 Z2; do ./build_tb.sh
$c; ./run_elfs.sh work/$c/obj/Vtb_top ../cv32e40p-fpu-perf/build/zfinx.elf; done
```

```
# area and delay, one configuration and flow per call (about 1-3 minutes each) cd ../cv32e40p-fpu-perf
LIB=/path/to/sky130_fd_sc_hd__tt_025C_1v80.lib ./synth/synth.sh F1 retime
```

We checked these public scripts against our records:

- F1 and F2 (with the #1065 patch) printed exactly the recorded cycle counts.
- synth.sh F1 retime printed exactly the recorded area (313,762.17 μm^2) and path (17.1214 ns).

The upstream matmul_32b_float row needs the testbench's own program build (with PULP added to its defines on F, so that its fp_enable() runs); that rebuild is not scripted here.

7 Limits

- A synthesis estimate, not an implementation. One typical corner, ideal clock, no wires, no clock tree, no place and route.
- The retimed netlists are not equivalence-checked.
- Retiming also shortens integer paths (I0 goes from 13.89 to 11.76 ns), so not all of the retimed gain comes from the FPU. With retiming, F2 is 12% above the integer core's own floor. Where the critical path sits after retiming was not recorded.
- A small, narrow workload set. Six small kernels and one upstream program, one compiler with default tuning, single-cycle testbench memory, no interrupts. Real code, slower memory and a latency-aware compiler will move the cycle numbers.
- Zfinx code size and register pressure are under-represented. The Zfinx kernels use the same instruction order as F, so they do not show the register-pressure cost that larger Zfinx code would pay.
- This is about performance and area only. It says nothing about verification effort or about the correctness of the pipelined FPU beyond the checksums of these programs.

8 Files

- tests/cv32e40p-fpu-perf/: kernels (src/kernels.c, src/crt0.S), build.sh, link.ld, README.md, PREDICTIONS.md
- tests/cv32e40p-fpu-perf/synth/: synth.sh, clock_gate_transparent.sv
- tests/cv32e40p-tb-run/: build_tb.sh, run_elfs.sh
- patches/tb_fpu_config_plumbing.patch